

Documentation Technique Complète

Plateforme de gestion d'écoles de mémorisation du Coran

Projet	TAHFIDZ SaaS — Application web multi-tenant
Version	1.0 — Mai 2026
Framework	Next.js 15 · App Router · TypeScript
Base de données	PostgreSQL (Supabase) · Prisma ORM
Auteur	Abdenour — he.abdenour@gmail.com
Pages	54 pages / routes · 34 APIs REST · 26 modèles Prisma

■ Table des matières

1. Vue d'ensemble du projet
2. Architecture technique
3. Modèle de données (Prisma)
4. API REST — Routes et endpoints
5. Profils utilisateurs
6. Fonctionnalités par profil
7. Internationalisation (FR/EN/AR)
8. Système de mémorisation
9. Gamification — Étoiles & Badges
10. Certificats personnalisables
11. Statistiques & Graphiques
12. Structure des fichiers
13. Variables d'environnement
14. Mise en production

1. Vue d'ensemble du projet

TAHFIDZ est une application web SaaS (Software as a Service) multi-tenant conçue pour la gestion complète d'écoles de mémorisation du Coran (tahfidz). Elle permet à chaque établissement de gérer ses élèves, enseignants, parents, groupes, présences, évaluations et la progression dans la mémorisation des 114 sourates du Coran.

Caractéristiques principales

- **Multi-tenant** — Chaque école est isolée par schoolId — données et configuration séparées
- **4 rôles** — SUPERADMIN · ADMIN · TEACHER · PARENT · STUDENT
- **Trilingue** — Interface complète en Français, English et ■ ■ ■ ■ ■ ■ ■ ■ (RTL)
- **54 pages** — Application complète avec tableaux de bord, listes, profils, formulaires
- **34 APIs REST** — Routes sécurisées par rôle avec validation Zod
- **26 modèles Prisma** — Schéma relationnel complet sur PostgreSQL

Problème résolu

Les écoles de tahfidz gèrent traditionnellement la progression des élèves sur papier ou via des feuilles Excel. TAHFIDZ remplace ce flux par une plateforme numérique qui offre : visibilité en temps réel, communication école–famille, traçabilité complète, et certificats imprimables selon le niveau atteint.

2. Architecture Technique

Stack technologique

Couche	Technologie	Version	Rôle
Frontend	Next.js	15.1.0	Framework React full-stack, App Router
Language	TypeScript	5.x	Typage strict
Styles	Tailwind CSS	3.x	Utility-first CSS, dark mode, RTL
ORM	Prisma	5.22.0	Schéma, migrations, client typé
Base de données	PostgreSQL	16	Hébergée sur Supabase
Auth	NextAuth v5	5.0.0-beta.25	JWT sessions, Prisma adapter
Validation	Zod	3.25	Schémas de validation serveur/client
Formulaires	react-hook-form	7.75	Formulaires performants
Graphiques	Recharts	3.8	Charts interactifs
Emails	Resend	6.x	Emails transactionnels
Queue	BullMQ + Redis	5.x	Tâches asynchrones
Icônes	Lucide React	1.14	Bibliothèque d'icônes SVG

Pattern Server → Client Components

L'app suit le pattern Next.js App Router : les Server Components fetcher les données depuis Prisma et les transmettent comme props aux Client Components qui gèrent l'interactivité et les traductions.

```
// Server Component (page.tsx)
export default async function Page() {
  const data = await prisma.student.findMany({ where: { schoolId } })
  return
}

// Client Component (*Client.tsx)
"use client"
export function StudentsListClient({ students }) {
  const { locale } = useLanguage() // i18n
  // render avec traductions
}
```

3. Modèle de Données — Schéma Prisma

Le schéma comprend 26 modèles organisés en 6 groupes fonctionnels :

Multi-Tenant Core

School	Entité racine : name, slug, plan (FREE/PRO), logo, address, settings JSON
User	Compte unifié : email, password (bcrypt), role, schoolId (tenant)
Session	Sessions NextAuth avec token et expiration
SchoolRequest	Demandes d'inscription de nouvelles écoles

Profils utilisateurs

Admin	Lié à User — accès complet à l'école
Teacher	Specialization, groupes assignés, évaluations
Parent	Comptes parents liés à plusieurs élèves
Student	dateOfBirth, enrollmentDate, totalStars, groupId, teacherId
ParentStudentLink	Table de jonction parent↔élève avec date de liaison

Pédagogie

Group	name, level (BEGINNER/INTERMEDIATE/ADVANCED), maxCapacity, isActive
Surah	id (1-114), name, nameAr, nameFr, totalVerses
MemorizationProgress	studentId × surahId × status × fromVerse × toVerse
StatusHistory	Historique de chaque changement de statut de mémorisation
MemorizedSurah	Sourates entièrement mémorisées — snapshot final

Évaluation & Présence

Evaluation	score (0-100), quality, decision (APPROVED/REJECTED/NEEDS_WORK), teacherId
Attendance	status (PRESENT/ABSENT/LATE/EXCUSED), sessionId, groupId
Exam	Modèle d'examen formel (phase 2)
StudentStats	Agrégats mis en cache : totalSurahs, avgScore, attendanceRate

Gamification

Badge	name, nameAr, description, icon, condition JSON, threshold
StudentBadge	Liaison étudiant↔badge avec date d'obtention

StarsLog	Log de chaque attribution d'étoile avec raison
-----------------	--

Communication

Announcement	title, content, type (GENERAL/URGENT/EVENT/ACHIEVEMENT), isPinned
GroupAnnouncement	Ciblage d'annonce par groupe
Notification	Notifications in-app par utilisateur, isRead
DirectMessage	Messagerie directe entre utilisateurs
AuditLog	Traçabilité de toutes les actions sensibles

4. API REST — Routes et Endpoints (34 routes)

Méthode	Endpoint	Rôle requis	Description
GET/POST	/api/students	ADMIN	Liste paginée / créer élève
GET/PUT/DELETE	/api/students/[id]	ADMIN/TEACHER	Profil, modifier, supprimer
PUT	/api/students/[id]/status	ADMIN	Activer/désactiver compte
POST	/api/students/[id]/transfer	ADMIN	Transférer vers autre groupe
GET	/api/students/export	ADMIN	Export CSV des élèves
GET/POST	/api/teachers	ADMIN	Liste / créer enseignant
GET/POST	/api/parents	ADMIN	Liste / créer parent
GET/PUT/DELETE	/api/parents/[id]	ADMIN	Profil parent
POST	/api/parent/link	PARENT	Lier un enfant par email
GET/POST	/api/groups	ADMIN	Liste / créer groupe
GET/PUT/DELETE	/api/groups/[id]	ADMIN/TEACHER	Détail groupe
GET/POST	/api/evaluations	TEACHER	Liste / créer évaluation
GET/PUT/DELETE	/api/evaluations/[id]	TEACHER	Modifier évaluation
GET/POST	/api/attendance	TEACHER	Pointage présences
GET	/api/attendance/export	ADMIN	Export présences Excel
GET/POST	/api/progress	TEACHER	Suivi mémorisation
PUT	/api/progress/[id]	TEACHER	Changer statut sourate
GET/POST	/api/announcements	ADMIN/TEACHER	Liste / créer annonce
PUT/DELETE	/api/announcements/[id]	ADMIN	Modifier / supprimer
GET/POST	/api/notifications	ALL	Notifications utilisateur
PUT	/api/notifications/[id]	ALL	Marquer comme lu
GET/POST	/api/admins	SUPERADMIN	Gestion des admins
PUT/DELETE	/api/admins/[id]	SUPERADMIN	Modifier admin
POST	/api/register-school	PUBLIC	Inscription nouvelle école
GET/PUT	/api/admin/schools	SUPERADMIN	Toutes les écoles
POST	/api/admin/school/logo	ADMIN	Upload logo école
GET/PUT	/api/admin/certificate-templates	ADMIN	Templates certificats JSON
GET/PUT	/api/profile	ALL	Profil personnel

PUT	/api/profile/password	ALL	Changer mot de passe
GET	/api/surahs	ALL	Liste des 114 sourates
GET/POST	/api/messages	ALL	Messagerie directe
GET/POST	/api/exams	TEACHER	Gestion examens
GET	/api/display	PUBLIC	TV Display (tableau école)
ALL	/api/auth/[...nextauth]	PUBLIC	NextAuth endpoints OAuth

5. Profils Utilisateurs & Accès

Rôle	Accès principal	Restrictions
SUPERADMIN	Toutes les écoles, création d'admins, statistiques globales	Aucune — accès plateforme complet
ADMIN	Toute son école : élèves, enseignants, parents, groupes, listes	Limité à son schoolId
TEACHER	Ses groupes, présences, évaluations, progression élèves	Données de ses propres groupes
PARENT	Profil et progression de ses enfants liés	Lecture seule sur les données enfant
STUDENT	Son propre profil, progression, badges, certificat	Données personnelles uniquement

Authentification

NextAuth v5 avec Prisma Adapter. Sessions JWT stockées en base. Le middleware Next.js vérifie le token sur chaque requête protégée et redirige vers /login si non authentifié ou si le rôle est insuffisant.

```
// middleware.ts
export { auth as middleware } from "@/auth"

// auth.ts - configuration NextAuth
auth({
  adapter: PrismaAdapter(prisma),
  session: { strategy: "jwt" },
  callbacks: {
    jwt({ token, user }) {
      if (user) { token.role = user.role; token.schoolId = user.schoolId }
      return token
    },
    session({ session, token }) {
      session.user.role = token.role
      session.user.schoolId = token.schoolId
      return session
    }
  }
})
```

6. Fonctionnalités par Profil

■■ Administrateur (Admin)

- Dashboard KPI : élèves actifs, taux de présence, annonces récentes, actions rapides
- Gestion des élèves : liste filtrée/paginée, profil complet, activation/désactivation, export CSV
- Gestion des enseignants : annuaire, groupes assignés, statistiques d'évaluations
- Gestion des parents : liens parent-enfant, historique
- Groupes : création, niveau (Débutant/Intermédiaire/Avancé), capacité, enseignant assigné
- Présences : vue par groupe et période (jour/semaine/mois/personnalisé), export
- Évaluations : liste filtrée par enseignant, élève, sourate, score
- Statistiques : graphiques recharts — scores mensuels, top élèves, occupation groupes, taux présence
- Certificats : éditeur de templates par niveau avec aperçu temps réel
- Annonces : 4 types (Général/Urgent/Événement/Réussite), épinglage
- Paramètres : apparence (dark/light/langue), logo école, profil administrateur

■■■ Enseignant (Teacher)

- Dashboard : résumé ses groupes, élèves actifs, prochaines actions
- Mes groupes : liste, détail par groupe avec tous les élèves et leur statut
- Présences : pointage rapide par session (Présent / Absent / En retard / Excusé)
- Évaluations : noter un élève (score 0-100, qualité, décision d'approbation)
- Progression : changer le statut de mémorisation sourate par sourate
- Profil élève : vue complète — mémorisations, badges, présences, évaluations

■ Élève (Student)

- Dashboard : étoiles totales, sourates mémorisées, statut actuel de mémorisation
- Ma progression : liste des sourates avec statut coloré et actions disponibles
- Mes badges : collection des badges gagnés avec date d'obtention
- Mes présences : historique des sessions avec codes couleur par statut
- Mon certificat : génération et impression du certificat PDF personnalisé

■■■■■ Parent

- Dashboard : résumé des enfants liés, dernières activités
- Profil enfant : progression mémorisation, dernières évaluations, présences
- Liaison enfant : lier un compte enfant par adresse email
- Notifications : alertes en temps réel (absences, évaluations...)

7. Internationalisation (FR / EN / AR)

L'application est entièrement traduite dans 3 langues avec support RTL pour l'arabe. Le système est basé sur un React Context personnalisé — pas de bibliothèque externe.

Architecture i18n

- **LanguageContext** — Fournit locale (fr|en|ar), setLocale() et dir (ltr|rtl)
- **LanguageProvider** — Wrapper dans chaque layout (admin, teacher, parent, student)
- **useLanguage()** — Hook consommé par tous les Client Components
- **localStorage** — Persistance du choix de langue entre sessions
- **document.documentElement** — dir et lang mis à jour immédiatement au changement

```
// Chaque composant client traduit son contenu
const { locale } = useLanguage()
const L = locale as "fr" | "en" | "ar"

const T = {
  title: { fr: "Élèves", en: "Students", ar: "الطلاب" },
  add: { fr: "Ajouter", en: "Add", ar: "إضافة" },
}
const t = (k: keyof typeof T) => T[k][L] ?? T[k].fr
```

TopBar Switcher

Un composant TopBar est présent sur toutes les pages (Admin, Teacher, Student). Pour Parent, les contrôles sont intégrés dans la navigation. Le switcher affiche **FR / EN / AR** et un bouton **dark mode**.

8. Système de Mémorisation du Coran

Chaque élève peut avoir un enregistrement MemorizationProgress pour chacune des 114 sourates. Le suivi est individuel, verset par verset, avec un historique complet des changements.

Cycle de statuts

Statut	Description	Acteur
IN_PROGRESS	Élève en cours de mémorisation	Enseignant
READY_FOR_RECITATION	Élève prêt à réciter devant l'enseignant	Élève
PENDING_TEACHER_APPROVAL	En attente que l'enseignant valide	Enseignant
UNDER_REVIEW	Enseignant en cours d'écoute/révision	Enseignant
NEEDS_REVISION	Des corrections sont nécessaires	Enseignant
MEMORIZED	Sourate entièrement mémorisée ✓	Enseignant

Données enregistrées par progression

- surahId — référence à la sourate (1-114)
- fromVerse / toVerse — plage de versets mémorisés
- status — état actuel dans le cycle
- StatusHistory — historique de chaque changement avec timestamp
- MemorizedSurah — snapshot quand status = MEMORIZED

9. Gamification — Étoiles & Badges

Système d'étoiles

Des étoiles sont attribuées à chaque évaluation réussie. Le cumul est stocké dans `Student.totalStars` et dans le log `StarsLog` avec la raison (évaluation, mémorisation...).

Système de badges

Les badges sont définis par l'administrateur avec des conditions JSON (threshold). Exemples de conditions :

Badge	Condition	Exemple
Premier pas	1 sourate mémorisée	<code>memorizedSurahs >= 1</code>
5 sourates	5 sourates mémorisées	<code>memorizedSurahs >= 5</code>
Étoile filante	50 étoiles cumulées	<code>totalStars >= 50</code>
Assidu	30 présences consécutives	<code>attendanceStreak >= 30</code>
Excellence	Score moyen > 90	<code>avgScore >= 90</code>

10. Certificats Personnalisables

L'administrateur peut personnaliser les templates de certificats pour chacun des 3 niveaux (Débutant, Intermédiaire, Avancé). Les templates sont stockés dans un fichier JSON.

Champs personnalisables par niveau

Champ	Description
title	Titre principal du certificat (ex. : Certificat de Mémorisation)
titleAr	Titre en arabe (ex. : ■■■■■■■■■■ ■■■■■■■■■■)
subtitle	Sous-titre / niveau (ex. : Niveau Débutant)
bodyText	Texte de description des accomplissements
arabicVerse	Verset ou texte arabe affiché en calligraphie
primaryColor	Couleur principale (bandeau, textes accentués)
accentColor	Couleur secondaire (dégradés)
lightColor	Fond clair pour les blocs
textColor	Couleur du texte sombre
badgeEmoji	Emoji représentant le niveau (■ / ■ / ■)

L'éditeur propose un aperçu temps réel. Le certificat final est généré en PDF via le navigateur (window.print()) depuis la page /admin/students/[id]/certificate.

11. Statistiques & Graphiques

La page Statistiques (admin/stats) agrège les données de toute l'école et les présente via Recharts.

Graphique	Type	Données affichées
Scores d'évaluation	LineChart	Score moyen + taux d'approbation — 6 derniers mois
Répartition mémorisation	PieChart (donut)	Distribution par statut (MEMORIZED, IN_PROGRESS...)
Présences globales	PieChart (donut)	PRESENT / ABSENT / LATE / EXCUSED + taux global
Top 10 sourates	BarChart horizontal	Sourates les plus mémorisées dans l'école
Top 5 élèves	Liste classée	Meilleurs élèves par nb d'étoiles et sourates
Occupation groupes	BarChart vertical	Nombre d'élèves par groupe

12. Structure des Fichiers (src/)

```
src/
■■■ app/
■   ■■■ admin/           # 20+ pages admin (students, teachers, parents, groups...)
■   ■■■ teacher/         # 8 pages enseignant
■   ■■■ student/         # 5 pages élève
■   ■■■ parent/          # 4 pages parent
■   ■■■ api/             # 34 Route Handlers REST
■   ■■■ login/           # Page authentification
■   ■■■ register-school/ # Inscription nouvelle école
■   ■■■ page.tsx         # Landing page publique
■
■■■ components/
■   ■■■ admin/           # *Client.tsx – composants traduits admin
■   ■■■ teacher/         # *Client.tsx – composants traduits enseignant
■   ■■■ student/         # *Client.tsx – composants traduits élève
■   ■■■ parent/          # *Client.tsx – composants traduits parent
■   ■■■ layout/          # Sidebars, TopBar, TopBarControls, ParentNav
■
■■■ contexts/
■   ■■■ LanguageContext.tsx # Système i18n maison (FR/EN/AR)
■
■■■ lib/
■   ■■■ prisma.ts         # Client Prisma singleton
■   ■■■ utils.ts          # formatDate, shortId, cn...
■   ■■■ i18n/
■       ■■■ pageTranslations.ts # Constantes de traduction partagées
■
■■■ auth.ts               # Configuration NextAuth v5
■■■ middleware.ts         # Protection des routes par rôle
```


13. Variables d'Environnement (.env)

Variable	Description	Exemple
DATABASE_URL	URL PostgreSQL Supabase (pooler)	postgresql://...@pooler.supabase.com/postgres
DIRECT_URL	URL directe PostgreSQL (migrations)	postgresql://...@db.supabase.com/postgres
NEXTAUTH_SECRET	Secret JWT NextAuth	openssl rand -base64 32
NEXTAUTH_URL	URL de l'application	http://localhost:3000
RESEND_API_KEY	Clé API Resend (emails)	re_xxxxxxxxxxxx
REDIS_URL	URL Redis (BullMQ)	redis://localhost:6379
NEXT_PUBLIC_APP_URL	URL publique (côté client)	https://tahfidz.app

14. Mise en Production

Étapes de déploiement

1. Clone & install

```
git clone && cd tahfidz && npm install
```

2. Environnement

```
Copier .env.example → .env et remplir toutes les variables
```

3. Migrations DB

```
npx prisma migrate deploy && npx prisma generate
```

4. Seed initial

```
npx prisma db seed (crée SuperAdmin + école démo)
```

5. Build

```
npm run build
```

6. Démarrage

```
npm start (ou déployer sur Vercel / Railway)
```

Recommandations

- Hébergement : Vercel (Next.js optimisé) ou Railway (Node + PostgreSQL)
- Base de données : Supabase (PostgreSQL managé, Storage pour logos)
- Redis : Upstash (serverless Redis) ou Railway Redis addon
- Emails : Resend (10 000 emails/mois gratuit)
- Domaine : Configurer NEXTAUTH_URL avec le domaine de production
- HTTPS : Obligatoire pour NextAuth (cookies sécurisés)

TAHFIDZ SaaS — Documentation générée le 17/05/2026

Abdenour · he.abdenour@gmail.com